

Guia de presentacion - Week 02

Grupo: Salazar D. - Ochoa A. - Chano J.

Objetivo del ejercicio: comparar arquitecturas de agentes en el mismo Vacuum World usando una interfaz comun: percept -> action. La idea principal no es solo limpiar habitaciones, sino observar que aporta cada arquitectura: reaccion, memoria, meta, utilidad y LLM como politica textual.

Parte	Responsable	Contenido	Idea central
1	Daniel Salazar	Contexto, ambiente, agentes dados	Todos usan el mismo socket; cambia la arquitectura.
2	Ochoa A.	goal_based y utility_based	Memoria para metas; utilidad inmediata sin planificacion.
3	Chano J.	Tests, torneo, LLM, scorecard, cierre	La evidencia sostiene la comparacion.

Parte 1 - Daniel Salazar

Enfoque: Contexto del Vacuum World y agentes dados

Tiempo sugerido: 3 a 4 minutos

Que cubrir

- Abrir el notebook y presentar el objetivo general.
- Explicar el contrato percept -> action.
- Explicar que el percepto es (location, is_dirty_here).
- Mostrar que el ambiente no se modifico para favorecer ningun agente.
- Presentar simple_reflex y model_based como agentes dados.

Por que esta parte importa

- El experimento es justo porque todos los agentes actuan dentro del mismo mundo.
- La diferencia entre agentes esta en la politica y en la memoria interna, no en reglas distintas del ambiente.
- El agente reflexivo sirve como linea base porque reacciona sin recordar nada.
- El agente basado en modelo muestra el primer beneficio de la memoria: saber cuando detenerse.

Interpretacion

- simple_reflex puede limpiar, pero sigue moviendose porque no sabe que ya termino.
- model_based agrega estado interno, asi que puede devolver NoOp cuando cree que todo esta limpio.
- La memoria ayuda, pero tambien depende de creencias iniciales correctas.

Cierre de esta parte

Con esto dejamos fija la comparacion: mismo ambiente, mismo percepto y mismas acciones; lo que cambia es como decide cada agente.

Parte 2 - Ochoa A.

Enfoque: Implementacion de goal_based y utility_based

Tiempo sugerido: 4 a 5 minutos

Que cubrir

- Abrir starter.py y mostrar goal_based.
- Explicar el estado interno: clean y loc.
- Mostrar que la meta es (False, False): ambas habitaciones limpias.
- Abrir utility_based y explicar la decision greedy.
- Conectar la utilidad con los costos dados por tournament.py.

Por que lo hicimos asi

- goal_based necesita comparar el estado conocido contra una meta explicita.
- No usamos una tabla de perceptos porque crece con el historial y no es una solucion compacta.
- utility_based no se convirtio en planificador porque la consigna pedia utilidad inmediata.
- Si el agente de utilidad planificara a futuro, el resultado perderia el punto del ejercicio.

Interpretacion

- goal_based limpia y se detiene porque su objetivo esta representado directamente.
- utility_based limpia si ve suciedad, pero no cruza por curiosidad si el costo inmediato no paga.
- Que deje suciedad en algunos casos no es un error: es consecuencia de optimizar una utilidad miope.

Cierre de esta parte

Las dos implementaciones muestran decisiones distintas: una persigue una meta y la otra optimiza utilidad inmediata.

Parte 3 - Chano J.

Enfoque: Evidencia, LLM, scorecard y conclusiones

Tiempo sugerido: 4 a 5 minutos

Que cubrir

- Correr python test_agents.py.
- Correr python starter.py para mostrar el torneo.
- Explicar por que el torneo usa 8 configuraciones.
- Mostrar parse_action y llm_agent como conexion del LLM.
- Mencionar que aicourse viene del ZIP base de Week 0.
- Cerrar con el scorecard y las conclusiones.

Por que lo hicimos asi

- Los tests se usan como evidencia de comportamiento, no solo como formalidad.
- El torneo usa la misma metrica para todos: utilidad neta.
- El LLM entra al mismo socket percept -> action para que la comparacion sea equivalente.
- El parser, retry, fallback y cache existen porque el LLM devuelve texto, no simbolos limpios.
- La corrida LLM se capo a 3 configuraciones porque Ollama local fue lento, que la consigna permite.

Interpretacion

- goal obtiene el mejor total clasico porque limpia y deja de gastar acciones.
- utility queda mas bajo porque deja valor sin capturar al no planificar.
- El LLM puede producir acciones validas, pero necesita infraestructura adicional para ser evaluable.
- La conclusion general es que la arquitectura y la metrica determinan el comportamiento observado.

Cierre de esta parte

Los resultados no se basan en intuicion: salen de tests, torneo, cache y scorecard.

Checklist para la presentacion

- Abrir el notebook week02_salazarD_ochoaA_chanoJ.ipynb.
- Mostrar starter.py cuando se hable de los agentes implementados.
- Ejecutar python test_agents.py para validar.
- Ejecutar python starter.py para mostrar el torneo.
- Usar llm_results.md para los resultados del LLM capado.
- Mantener el foco en la comparacion de arquitecturas, no en memorizar codigo.

Comandos rapidos

Accion	Comando
Tests	python test_agents.py
Torneo	python starter.py
Notebook	jupyter notebook week02_salazarD_ochoaA_chanoJ.ipynb
Doctor LLM	\$env:PYTHONIOENCODING='utf-8'; \$env:AICOURSE_MODEL='qwen2.5:1.5b'; python -m aicourse.doctor